

Map Optimization _ Instanced Static Meshes Evaluation



svc.glo.c2spmig

Published 7/25/2025

► Table of Contents

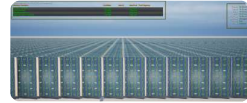
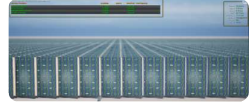
Overview

We want to evaluate UE5 instancing on static meshes since the original DL1 heavily relies on repeated meshes with material/texture swap (what is called a skin in DL1).

Instanced Static Mesh (**ISM**) component helps efficiently group identical static meshes into one actor.

Instanced Static Meshes should be able to reduce memory, reduce draw calls on CPU and also improve GPU performance thanks to the batching. It might however suffer sometimes from the non support of occlusion to discard some parts and might draw more elements that necessary.

Performance comparison from UE5 reference documents

	Static Mesh Component	Instanced Static Mesh Component
Screenshot		
ViewState memory	1.13MB	1.13MB
Primitive memory	11.79MB	0MB
Scene memory	1.59MB	0.90MB
Game	10.10ms	6.65ms
GPU	13.50ms	17.31ms
Draw calls	658	201
Primitives	1968200	10100
Light interaction memory	0MB	0MB

UE5 Reference documents

- [Instanced Static Mesh Component in Unreal Engine | Unreal Engine 5.5 Documentation | Epic Developer Community](#)

- [Valley of the Ancient Sample Game for Unreal Engine | Unreal Engine 5.5 Documentation | Epic Developer Community](#)

How to create ISM in UE5

There are two way to create ISM in UE5:

- **Packed Level Actor**
- **Harvest Instances Tool**

The **Slums** map has the same characteristics as the Valley of the Ancient. it is a large map with multiple repetitive static mesh Actors. The valley of the ancient is using ISM through PackedLevelActor.

Packed Level Actor

To create a packed level actor, follow these steps:

1. Select the desired actors.
2. Right-click the selection to open the context menu and click **Level > Create Packed Level Actor**.
3. Choose the pivot type and actor.
4. Click **Ok**.

- 5. Name the level and click **Save**.
- 6. Name the Blueprint and click **Save**.

Test with Slums Map mesh(Packed Level Actor)

We copied the bridge part from the `slums_submap` map to the new map to test the ISM function.we only set ISM for vehicles

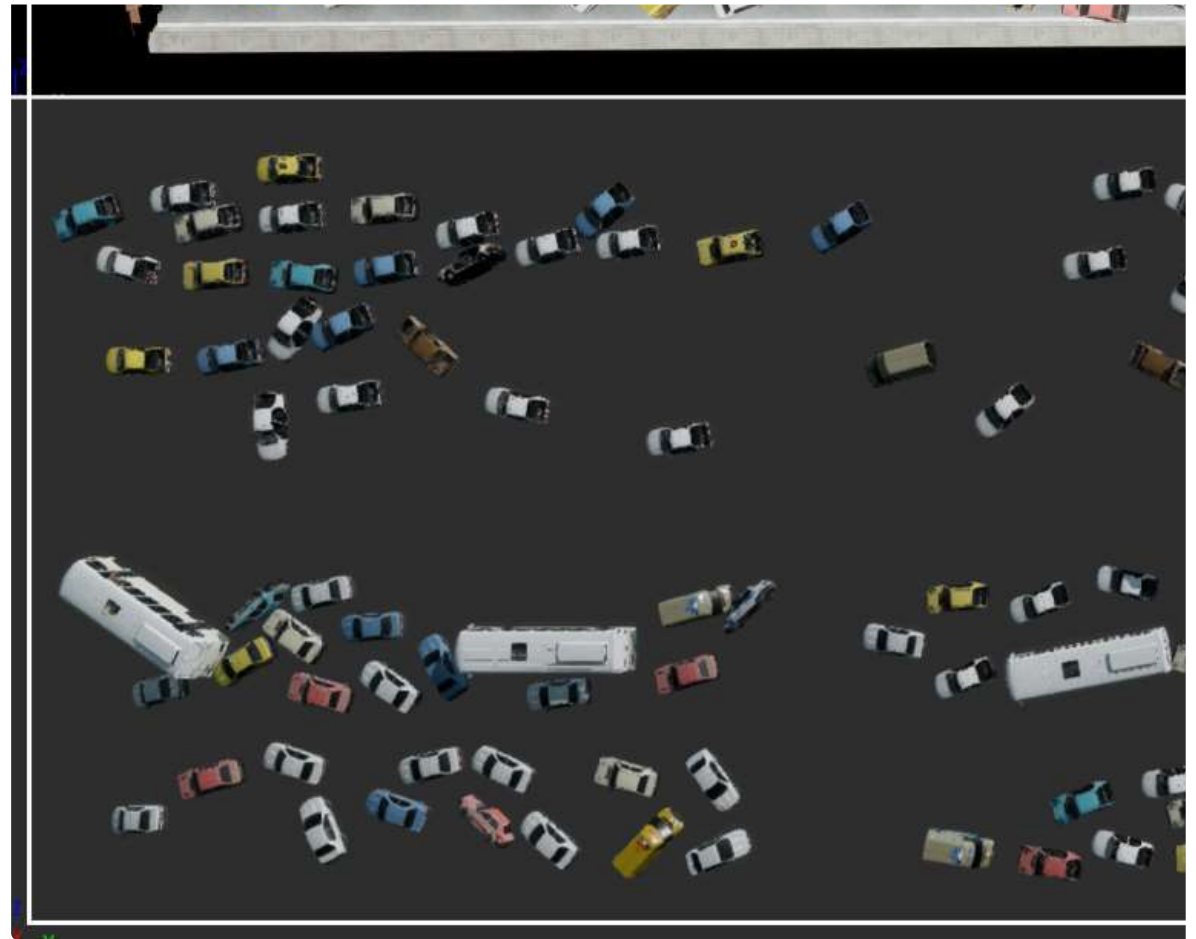


Bridge Scene evaluation / stat SceneMemory/Sc	
	Static Mesh Component Stats

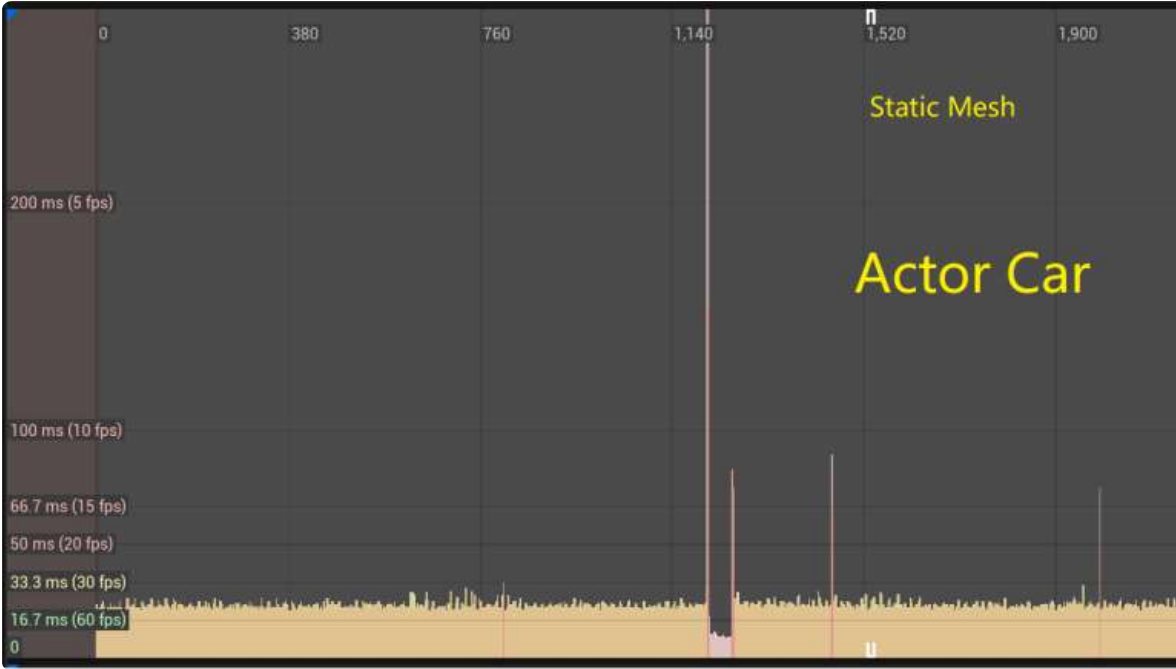
[illegible]

Countdown	Average	Min	Max
Empty queue (m)	101.66	101.66	249.00
Busy queue active instances	211.00	211.00	211.00
Busy queue total instances	211.00	211.00	211.00
Power Line	211.00	211.00	211.00
Light in action	2.00	2.00	2.00
Decade in action	0.00	0.00	0.00





ViewState (Mem)	0.03MB
Scene (Mem)	0.05MB
Mesh draw calls (avg)	336
RanderViewFamily (avg)	13.41ms
InitViews (avg)	6.32ms
OcclusionSubmittedFence Wait (avg)	0ms
DeferredShadingSceneRendererLighting (avg)	0.97ms
Finish Gather Ray Tracing Instances (avg)	0.03ms
Lighting drawing (avg)	0.19ms

Proj Shadow drawing (avg)	0.11ms
Mesh draw calls	336
Ray Tracing total instances	211
Insights	
Rendering Comparison	

Name	I.Max	I.Avg	I.Med	I.Min
CPU (0 / 25,011)	-∞	NaN	NaN	∞
▼ GPU (14,715)	24.76	NaN	NaN	0
Basepass	12.87	4.14	4.16	0
RenderVelocities	11.44	3.67	3.70	0
Postprocessing	13.01	3.15	3.15	0
TemporalSuperResolution	10.94	2.64	2.65	0
RenderDeferredLighting	8.21	2.29	2.24	0
ShadowDepths	7.40	2.16	2.17	0
SlateUI	4.61	1.57	1.54	0
LumenScreenProbeGather	3.30	1.09	1.07	0
Lights	1.69	0.43	0.43	0
NaniteBasePass	1.58	0.41	0.38	0
NaniteVisBuffer	1.37	0.39	0.36	0
ShadowProjection	1.38	0.32	0.31	0
Prepass	0.97	0.32	0.32	0
LumenReflections	1.46	0.31	0.32	0
Translucency	1.34	0.29	0.27	0
RayTracingScene	0.99	0.22	0.18	0
NaniteReadback	0.54	0.18	0.18	0
LumenSceneLighting	0.71	0.16	0.16	0
TranslucentLighting	1.56	0.12	0.13	0
MotionBlur	0.48	0.12	0.13	0
DistanceFields	1.18	0.06	0.06	0
BuildRenderingCommandsDeferred	0.17	0.05	0.05	0
HZB	0.25	0.05	0.05	0
GPUSceneUpdate	0.12	0.03	0.03	0
SortLights	0.28	0.02	0.02	0
Unaccounted - Frame: 16438	20.57	0.01	0	0
BeginOcclusionTests	0.22	0.01	0.01	0
Unaccounted - Frame: 16439	19.63	0.01	0	0
Unaccounted - Frame: 16385	24.76	0.01	0	0
Unaccounted - Frame: 16442	17.00	0.01	0	0
Unaccounted - Frame: 16382	17.71	0.01	0	0
Unaccounted - Frame: 16446	16.79	0.01	0	0
Unaccounted - Frame: 16443	15.97	0.01	0	0
Unaccounted - Frame: 15888	16.85	0.01	0	0
Unaccounted - Frame: 16444	17.24	0.01	0	0

Unaccounted - Frame: 16444	17.24	0.01	0	0
Unaccounted - Frame: 16247	17.20	0.01	0	0
Unaccounted - Frame: 16445	15.11	0.01	0	0
Unaccounted - Frame: 16918	16.66	0.01	0	0
Unaccounted - Frame: 15292	15.69	0.01	0	0
Unaccounted - Frame: 16119	16.17	0.01	0	0
Unaccounted - Frame: 16587	15.97	0.01	0	0
Unaccounted - Frame: 16440	16.62	0.01	0	0
Unaccounted - Frame: 17046	15.17	0.01	0	0

Basepass	4.14
RenderVelocities	3.67
Postprocessing	3.15
TemporalSuperResolution	2.64
RenderDeferredLighting	2.29
ShadowDepths	2.16
Insights file	\\CNSHASPTSANFSV1\Travis\6.Users\ChenShengge\ISM\OldCar.utrac

According to the table, we can draw the following conclusions:

- The screen data is jumping, so the data is accidental. through the comparison of insights, we can see that GPU will use more resources.
- ISM has the effect of optimizing memory, the memory is reduced from 0.05 to 0.02MB.
- Performance is not optimized. after changing to ISM, rendering threads increased their usage.

Harvest Instances Tool (For static mesh actors)

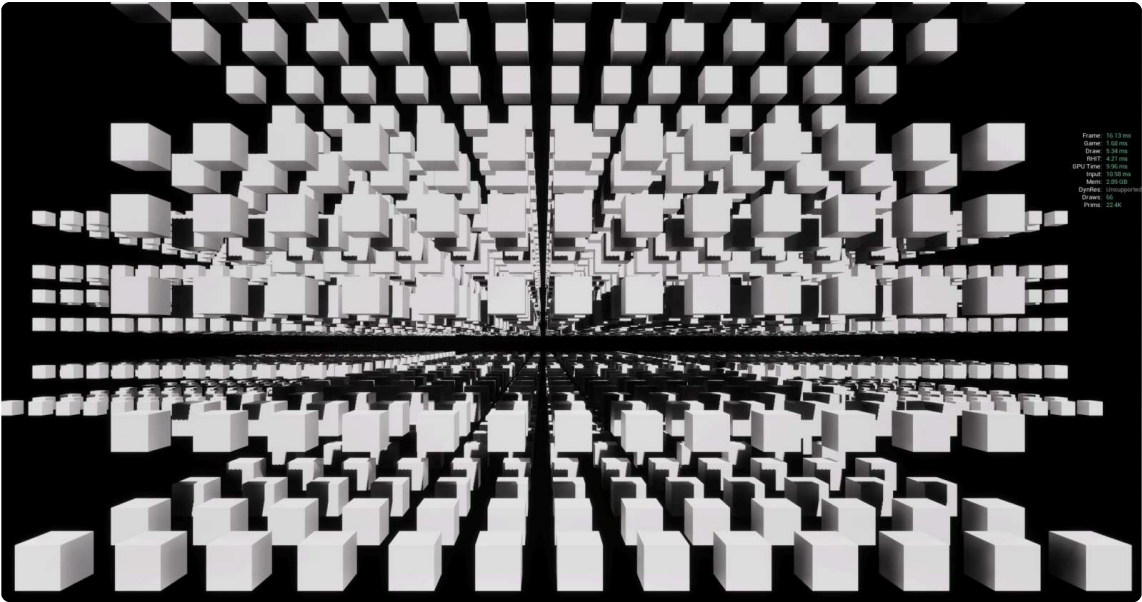
Suitable for multiple static Actor in the same area, similar to books on the shelf.

Use the Harvest Instances tool, follow these steps:

1. Switch to **Modeling mode** in UE5 editor
2. Select the static mesh actor to be processed
3. click **XForm > Harvest Instances**
4. Select **ISM** as the type, check **SingleActor**, change the name for the new actor
5. click **accept**.

Test with cube mesh(Harvest Instances Tool)

We believe that ISM can optimize the performance, and finally it is reflected in FPS. At present, the results are not satisfactory, so we use cubes to test.

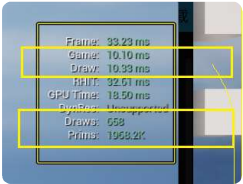
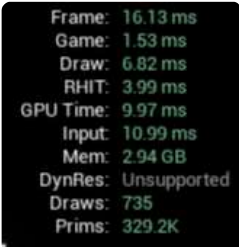
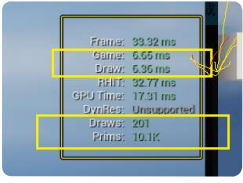
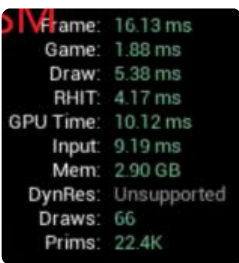


stat unit \ RTX3060

Test Map
/Game/Developers/chensheng
/Game/Developers/chensheng

Custom build \\10.72.51.252_CustomBuilds\20250427_Dev19576_Art19396_G

Game Mode	UE sample	In Game	
		first time	second time
Static Mesh Actors			

			
ISM			
Game	10.10->6.65	16.13->16.13	16.13->16.13
Draw	10.33->6.36	6.96->5.27	6.82->5.38
RHIT	32.6->32.7	4.15->4.18	3.99->4.17
Draws	658->201	633->67	735->66
Prims	1968.2->10.1	321.3->22.4	329.2->22.4

We added a lot of meshes, but the results didn't change much, so we think that the ISM function has no or little effect on performance, because it is impossible for us to have so many meshes in the

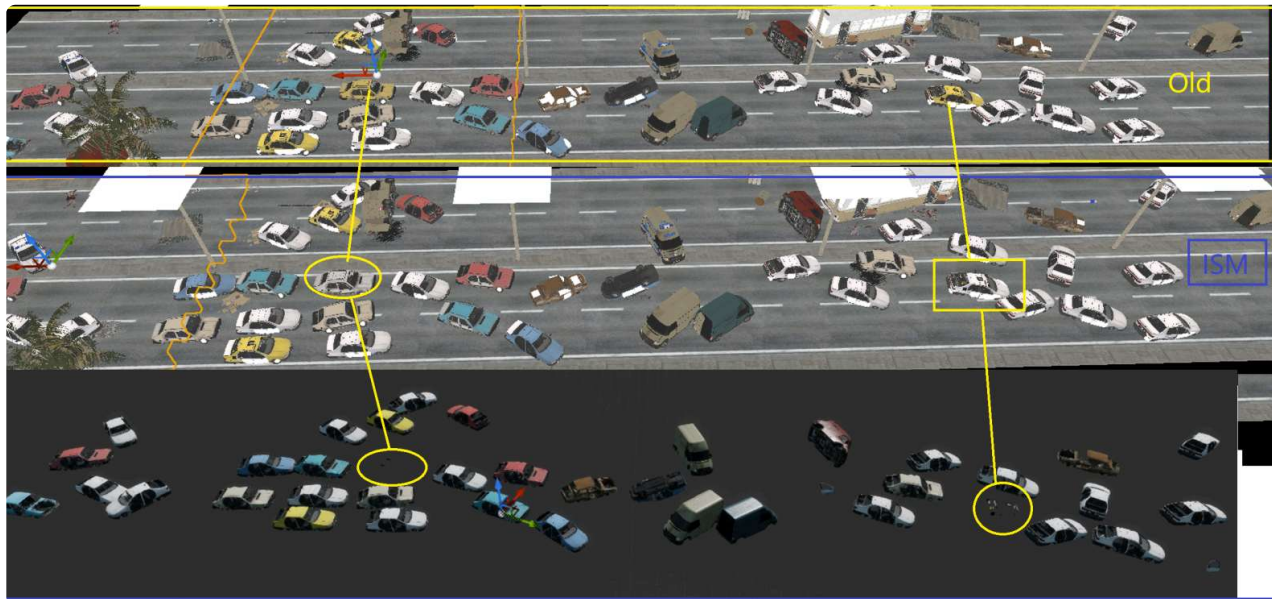
same area at the same time in the game.

Those are ultimate tests, that's either broken either that's not working as it used to

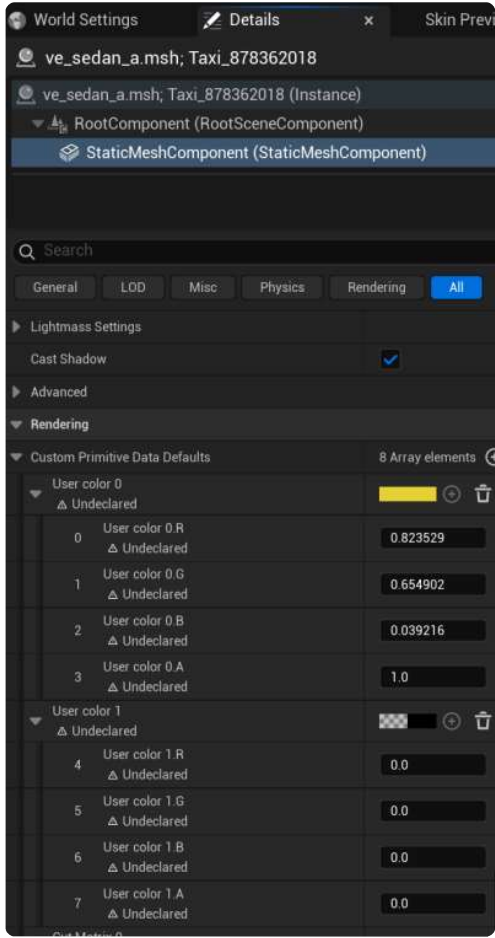
Per-instance material properties limitation

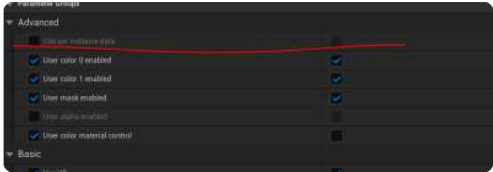
At the current stage of the test, we have to face the fact that our materials are not configured to be used with ISM. After we instantiate several identical static mesh, the color data will be invalid. TA has added the option of using color for materials.

When we use the material settings, other actor using this material will have display problems.



- Here are details related to that issue

Detail	Description
 A screenshot of the Unreal Engine Details panel for a StaticMeshComponent. The 'Rendering' tab is selected. Under 'Custom Primitive Data Defaults', there are two sections: 'User color 0' and 'User color 1'. 'User color 0' has four parameters: 'User color 0.R' (0.823529), 'User color 0.G' (0.654902), 'User color 0.B' (0.039216), and 'User color 0.A' (1.0). 'User color 1' has four parameters: 'User color 1.R' (0.0), 'User color 1.G' (0.0), 'User color 1.B' (0.0), and 'User color 1.A' (0.0).	• When using static mesh, we can achieve different color appearances by setting the Custom primitive data defaults parameters.
 A screenshot showing several instanced static meshes of a car model in a game view. The meshes are arranged in a row, and their positions are visible in the level.	• If packed level instance is

	used, this parameter will be invalid.
	• Using this material parameter, each ISM can have different material parameters
	• If this parameter is enabled, SM's custom color will no longer be applicable.

There are some possibilities to define some properties as unique to the instance while using ISM, so this will have to be integrated in the future.

Additional tests with increased count of meshes

Regarding our game and conducted tests in this page, all previous amount of meshes are too few. A map such as slums has 200.000+ static meshes, so it would be interesting to increase instance count.

Performance comparison with large count of meshes / PIE		
	StaticMeshes	Instanced Static Meshes
Static Unit		
Count	44.000	59.000
Frame	18.5ms	16.61ms
Game	10.06ms	8.68ms
Draw	18.23ms	16.26ms
RHIT	5.15ms	5.92ms
GPU Time	9.16ms	8.56ms

Mem	13.16GB	12.73GB
Draws	9571	253
Prims	2107000	4487

After increasing the count of meshes there are few remarks to be considered:

- Editor becomes very slow in the case of Static Meshes, frame time will drop to 130ms
- Running PIE has acceptable performance in both situations
- In order to speed up the test, we duplicated instances groups instead of duplicated instances within a group. This should have a negative impact on performance so with a better configuration, ISM should perform faster, by a bit.

That said, regarding numbers, it is clear that ISM has a clear advantage in terms of performance comparing to ISM. CPU, GPU and Memory are improved.

It would be interesting to test it on the full slums map.

Evaluate ISM with Old Town map

Context, setup and limitations

As this evaluation is done post-project, we are evaluating the Old Town with a from scratch importation using ISM and Static Meshes on submaps. This map is probably the largest in the game, it is exceeding slums actor count by 25%.

We had the possibility to bring the terrain with default material, foliage and water, but not to reconfigure the Time of Day and other refinements done by the TA Team. Lights were imported but all disabled since their intensity is off the charts because the sun intensity is too low compared to what artists configured in the slums map and should not really matter when it comes to compare performance difference between ISM vs Static Meshes.

Please also remember that this whole project is configuring all static meshes with nanite support except for those that have transparencies since UE 5.5 doesn't support transparency on Nanite.

ISM feature is added to the MapImporter as a preview and could be refined in the future if necessary, however, here are few notes about the current status:

- ISM doesn't have a per instance configuration, so there will be a material configuration difference between ISM and Static Meshes submaps. This is not an issue if materials are configured to handle few properties on the instance.
- ISM seems to bring trouble on lumen concerning raytracing. In some situations, the amount of registered meshes to contribute to the lighting is just too high. This part needs to be investigated further, it might be related to the lack of occlusion in the case of ISM. To be noted that in the editor, there are a lot of performance fluctuation and this is not always reproducible.
- ISM doesn't have a per instance collision setup and we don't know if this could impact the gameplay. From what we see, the old town is also prepared quite differently from the slums

and is using additional dummy box to help the edge grab feature so that might be an alternative setup to be considered.

On the editor side:

- Advantages of ISM
 - Fast to load
 - Fast to add a new instance
 - Fast to save the map
- Disadvantages of ISM
 - We are fully losing the naming convention and IDs from the original game, so this is hard for artist to localize and identify elements
 - Edition is painful, always need to go to a special subsection of the modeling and select the instance group before being able to select one instance and edit it

Old Town / Rendering comparison
ISM





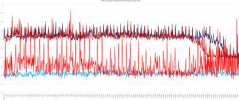
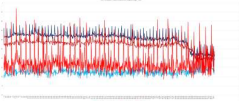
As we can see on the rendering, there are no difference except on materials.

Performance Evaluation

Configuration

- Tested in PIE in 1440P
- GPU nVidia RTX 3060 (12GB)
- CPU AMD Ryzen PRO 5945WX (12 cores)

Source Data		
Software Raytracing	ISM	UEDPIE_0_old_town_submap-FPS-WindowsEditor.csv
	Static Meshes	UEDPIE_0_old_town_non_ism_submap-FPS-WindowsEditor.csv
Hardware RaytracingISM	ISM	UEDPIE_0_old_town_submap-FPS-WindowsEditor.csv
	Static Meshes	UEDPIE_0_old_town_non_ism_submap-FPS-WindowsEditor.csv

Old Town / Performance overview between static		
Rendering	Graphs	
Software Raytracing		
Hardware Raytracing		



Old Town / Performance average over the 20 first seconds

Section	Software Raytracing		Hardware Raytracing		Comments
	ISM	Static Meshes	ISM	Static Meshes	
GT (ms)	11.08	15.78	12.40	17.70	ISM is overall quicker on CPU.
GPU (ms)	32.68	32.59	32.45	28.40	Gains on the GPU for Static Meshes with Hardware Raytracing is higher

This table is covering the window from 0 to 20s in order to avoid to bias results because of manual operations.

- Software raytracing perform better for ISM on the CPU side, while not having obvious impact on the GPU
- For Static Meshes, they will always impact the CPU, but this could be leveraged by the usage of HLODs. To be noted that when hardware raytracing is used, the GPU performance will be the

best with Static Meshes.

Conclusion

- ISM and Static Meshes performance need to be further tested with usage of HLODs
- Static Meshes or ISM seems to be mostly be a trade off between CPU/GPU time
- Need to integrate per instance properties
- Need to evaluate HLODs performance difference between ISM and Static Meshes.

^ Project Travis

✓ Art

✓ Cutscenes

✓ Design

✓ Engineering

✓ How to join us

✓ Lighting

- ✓ Production
- ✓ QA
- ✓ TA
- ✓ Techland Docs
- ✓ Test

